

LAPORAN PRAKTIKUM 6 STRUKTUR DATA

“Doubly Linked List”

Disusun Oleh:

Dinda Amelia

(2511531020)

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Muhammad Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG 2026

KATA PENGANTAR

Segala puji bagi Allah SWT yang telah memberikan kesempatan serta kemudahan kepada penulis untuk dapat menyelesaikan Laporan Praktikum pada mata kuliah Struktur Data, sehingga Laporan Praktikum ini dapat dikumpulkan dengan tepat waktu. Atas rahmat dan karunianya Laporan Praktikum dapat terselesaikan dengan baik. Sholawat serta salam juga penulis sampaikan kepada baginda tercinta yaitu Nabi Muhammad SAW, yang kita nantikan syafa'atnya di akhirat nanti. Laporan Praktikum ini bertujuan untuk menambah wawasan para pembaca untuk lebih memperdalam ilmu yang ada pada makalah ini.

Laporan pratikum ini disusun sebagai bentuk penanggung jawaban atas pelaksanaan kegiatan pratikum mata kuliah struktur data yang membahas tentang Doubly Linked List di Java Eclipse. Dalam praktikum ini, penulis tidak hanya belajar tentang Penyusunan laporan ini bertujuan untuk memahami konsep dasar Doubly Linked List.

Dalam penyusunan Laporan Praktikum ini, penulis mengalami banyak kesulitan dan penulis menyadari bahwa Laporan Praktikum ini masih jauh dari kesempurnaan. Untuk itu, penulis sangat mengharapkan kritik dan saran yang membangun demi kesempurnaan pada Laporan Praktikum ini.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I	1
PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Pratikum	1
1.3 Manfaat Praktikum	1
BAB II.....	3
PEMBAHASAN.....	3
2.1 Dasar Teori.....	3
2.2 Langkah kerja.....	4
2.2.1 Buka eclipse.....	4
2.2.2 Membuat Java Project	4
2.2.3 Class 1 “NodeDLL_2511531020”	5
2.2.4 Class 2 “InsertDLL_2511531020”	7
2.2.5 Class 3 “PenelusuranDLL_2511531020”	11
2.2.6 Class 4 “HapusDLL_2511531020”	14
2.3 Tugas Praktikum.....	19
2.3.1 Class 1 “Lagu_2511531020”	19
2.3.2 Class 2 “Musik_2511531020”	22
BAB III.....	29
KESIMPULAN	29
DAFTAR PUSTAKA.....	30

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep fundamental dalam ilmu komputer yang berfungsi untuk mengatur dan mengelola data secara efisien. Salah satu bentuk struktur data yang sering digunakan adalah Doubly Linked List (DLL), yaitu kumpulan node yang saling terhubung dua arah melalui pointer next dan prev. DLL memberikan fleksibilitas dalam proses penambahan, penghapusan, dan penelusuran data karena setiap simpul dapat diakses baik dari depan maupun dari belakang. Pemahaman mengenai DLL sangat penting bagi mahasiswa untuk menguasai teknik pengelolaan memori dinamis serta mendukung implementasi algoritma yang lebih kompleks.

1.2 Tujuan Pratikum

1. Memahami konsep dasar Doubly Linked List dan perbedaannya dengan Single Linked List.
2. Mempelajari cara membangun node dengan pointer next dan prev.
3. Mengimplementasikan operasi dasar pada DLL, seperti insert, delete, dan traversal.
4. Melatih kemampuan analisis dalam menghubungkan teori struktur data dengan implementasi program.

1.3 Manfaat Praktikum

1. Memberikan keterampilan praktis dalam mengelola data secara dinamis menggunakan DLL.
2. Membantu mahasiswa memahami bagaimana pointer bekerja dalam menghubungkan simpul dua arah.
3. Menjadi dasar untuk mempelajari struktur data yang lebih kompleks, seperti circular linked list atau tree.

4. Meningkatkan kemampuan problem solving dalam pemrograman dengan efisiensi memori dan fleksibilitas operasi.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Doubly Linked List (DLL) adalah salah satu bentuk struktur data dinamis yang terdiri dari rangkaian simpul (node). Setiap simpul memiliki tiga komponen utama:

1. Data → menyimpan nilai atau informasi.
2. Pointer next → menunjuk ke simpul berikutnya.
3. Pointer prev → menunjuk ke simpul sebelumnya.

Dengan adanya dua pointer (next dan prev), DLL dapat ditelusuri baik dari arah depan (head → tail) maupun dari arah belakang (tail → head).

Operasi Dasar pada DLL

1. Insert (penambahan simpul)
 - Menambahkan node di awal (head).
 - Menambahkan node di akhir (tail).
 - Menambahkan node di posisi tertentu.
2. Delete (penghapusan simpul)
 - Menghapus node di awal.
 - Menghapus node di akhir.
 - Menghapus node di posisi tertentu.
3. Traversal (penelusuran simpul)
 - Forward Traversal → menelusuri dari head ke tail menggunakan pointer next.
 - Backward Traversal → menelusuri dari tail ke head menggunakan pointer prev.

2.2 Langkah kerja

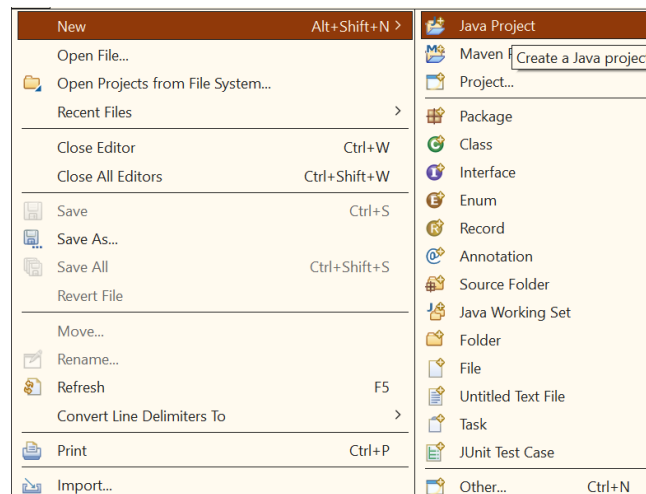
2.2.1 Buka eclipse



Gambar 2.1 Buka eclipse jalankan

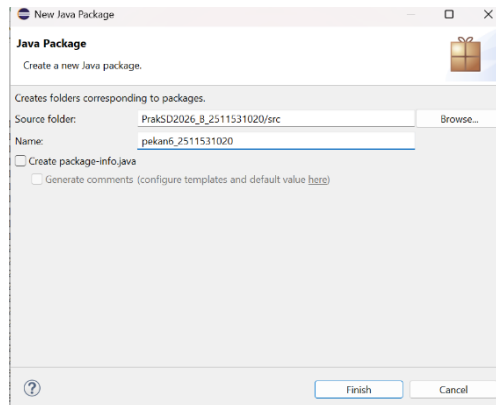
2.2.2 Membuat Java Project

1. Klik file
2. Klik java project



Gambar 2.2 Java Project

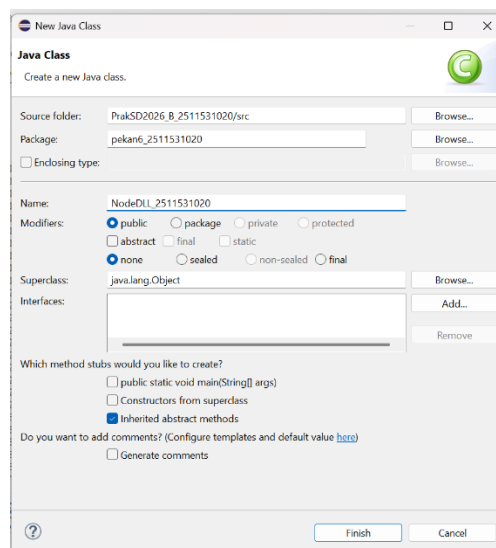
3. Buat nama project



Gambar 2.3 Nama Project

2.2.3 Class 1 “NodeDLL_2511531020”

1. Membuat package pekan6_2511531020 dan class NodeDLL_2511531020



Gambar 2.4 Membuat Nama package dan class NodeDLL_2511531020

2. Klik Finish
3. Kode Program
4. **package** pekan6_2511531020;
- 5.
6. **public class** NodeDLL_2511531020 {
7. //mendefenisikan kelas Node
8. **int** data_1020; // data

```

9.      NodeDLL_2511531020 next_1020; //Pointer ke next Node
10.     NodeDLL_2511531020 prev_1020; // Pointer ke previous
      node
11.
12.     //Konstruktor
13.     public NodeDLL_2511531020(int data_1020) {
14.         this.data_1020 = data_1020;
15.         this.next_1020 = null;
16.         this.prev_1020 = null;
17.     }
18. }

```

Gambar 2.5 Class 1 “NodeDLL_2511531020”

4. Analisis Langkah

a. Deklarasi Package

- package pekan6_2511531020; → Menunjukkan bahwa kelas ini berada dalam package tertentu sesuai NIM, sehingga memudahkan pengelompokan file program.

b. Deklarasi Kelas

- public class NodeDLL_2511531020 → Mendefinisikan kelas node yang akan digunakan dalam struktur Doubly Linked List (DLL).

c. Deklarasi Atribut

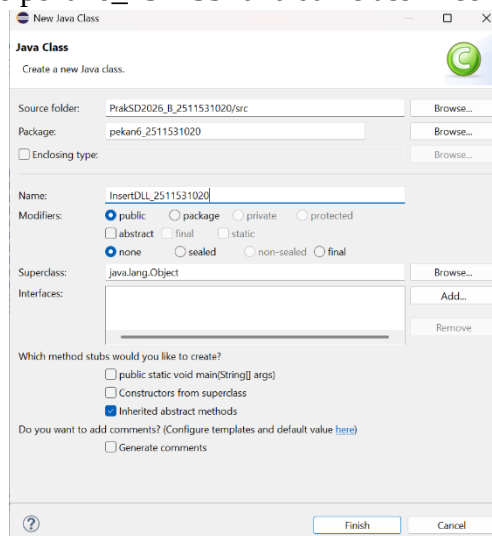
- int data_1020 → Menyimpan nilai/data dari node.
- NodeDLL_2511531020 next_1020 → Pointer ke node berikutnya.
- NodeDLL_2511531020 prev_1020 → Pointer ke node sebelumnya. → Dengan adanya next dan prev, node dapat dihubungkan dua arah (maju dan mundur).

d. Konstruktor

- `public NodeDLL_2511531020(int data_1020) →` Saat objek node dibuat, data langsung diinisialisasi sesuai input. `→ next_1020` dan `prev_1020` otomatis bernilai null, artinya node berdiri sendiri sebelum dihubungkan ke list.

2.2.4 Class 2 “InsertDLL_2511531020”

1. Membuat package `pekan6_2511531020` dan class `InsertDLL_2511531020`



Gambar 2.6 Membuat Nama package dan class `InsertDLL_2511531020`

2. Klik Finish

a. Kode Program

```

3. package pekan6_2511531020;
4.
5. public class InsertDLL_2511531020 {
6.     // menambahkan node di awal DLL
7.     static NodeDLL_2511531020
8.     insertBegin_1020(NodeDLL_2511531020 head_1020, int data_1020){
9.         NodeDLL_2511531020 new_node_1020 = new
10.        NodeDLL_2511531020(data_1020);
11.        new_node_1020.next_1020 = head_1020;
12.        if (head_1020 != null){
13.            head_1020.prev_1020 = new_node_1020;
14.        }
15.        return new_node_1020;
16.    }
17.    // fungsi menambahkan node di akhir

```

```

17.     public static NodeDLL_2511531020
insertEnd_1020(NodeDLL_2511531020 head_1020, int newData_1020){
18.         NodeDLL_2511531020 newNode_1020 = new
NodeDLL_2511531020(newData_1020);
19.         if (head_1020 == null){
20.             head_1020 = newNode_1020;
21.         } else {
22.             NodeDLL_2511531020 curr_1020 = head_1020;
23.             while (curr_1020.next_1020 != null){
24.                 curr_1020 = curr_1020.next_1020;
25.             }
26.             curr_1020.next_1020 = newNode_1020;
27.             newNode_1020.prev_1020 = curr_1020;
28.         }
29.         return head_1020;
30.     }
31.
32.     // fungsi menambahkan node di posisi tertentu
33.     public static NodeDLL_2511531020
insertAtPosition_1020(NodeDLL_2511531020 head_1020, int
pos_1020, int new_data_1020) {
34.         NodeDLL_2511531020 new_node_1020 = new
NodeDLL_2511531020(new_data_1020);
35.         if (pos_1020 == 1) {
36.             new_node_1020.next_1020 = head_1020;
37.             if (head_1020 != null) {
38.                 head_1020.prev_1020 = new_node_1020;
39.             }
40.             head_1020 = new_node_1020;
41.             return head_1020;
42.         }
43.         NodeDLL_2511531020 curr_1020 = head_1020;
44.         for (int i_1020 = 1; i_1020 < pos_1020 - 1 && curr_1020
!= null; ++i_1020) {
45.             curr_1020 = curr_1020.next_1020;
46.         }
47.         if (curr_1020 == null) {
48.             System.out.println("Posisi tidak ada");
49.             return head_1020;
50.         }
51.         new_node_1020.prev_1020 = curr_1020;
52.         new_node_1020.next_1020 = curr_1020.next_1020;
53.         curr_1020.next_1020 = new_node_1020;
54.         if (new_node_1020.next_1020 != null) {
55.             new_node_1020.next_1020.prev_1020 = new_node_1020;
56.         }
57.         return head_1020;
58.     }
59.
60.     public static void printList_1020(NodeDLL_2511531020
head_1020) {
61.         NodeDLL_2511531020 curr_1020 = head_1020;
62.         while (curr_1020 != null) {
63.             System.out.print(curr_1020.data_1020 + " <-> ");
64.             curr_1020 = curr_1020.next_1020;
65.         }
66.         System.out.println();

```

```

67.     }
68.
69.     public static void main(String[] args) {
70.         NodeDLL_2511531020 head_1020 = new
NodeDLL_2511531020(2);
71.         head_1020.next_1020 = new NodeDLL_2511531020(3);
72.         head_1020.next_1020.prev_1020 = head_1020;
73.         head_1020.next_1020.next_1020 = new
NodeDLL_2511531020(5);
74.         head_1020.next_1020.next_1020.prev_1020 =
head_1020.next_1020;
75.
76.         System.out.print("DLL Awal: ");
77.         printList_1020(head_1020);
78.
79.         head_1020 = insertBegin_1020(head_1020, 1);
80.         System.out.print("Simpul 1 ditambah di awal: ");
81.         printList_1020(head_1020);
82.
83.         System.out.print("Simpul 6 ditambah di akhir: ");
84.         int data_1020 = 6;
85.         head_1020 = insertEnd_1020(head_1020, data_1020);
86.         printList_1020(head_1020);
87.
88.         System.out.print("Tambah node 4 di posisi 4: ");
89.         int data2_1020 = 4;
90.         int pos_1020 = 4;
91.         head_1020 = insertAtPosition_1020(head_1020, pos_1020,
data2_1020);
92.         printList_1020(head_1020);
93.     }
94. }

```

Gambar 2.7 Class 2 “InsertDLL_2511531020”

4. Analisis Langkah

a. Deklarasi Package dan Class

- package pekan6_2511531020; → menandakan file berada dalam package sesuai NIM.
- public class InsertDLL_2511531020 → kelas ini berisi fungsi-fungsi untuk operasi pada Doubly Linked List (DLL).

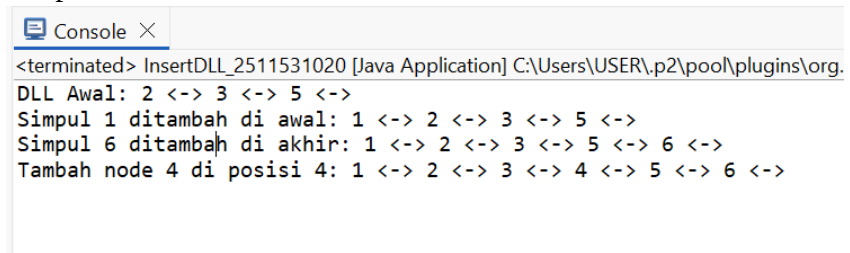
b. Fungsi insertBegin_1020

- Membuat node baru dengan data yang diberikan.

- Menjadikan node baru sebagai head, lalu menghubungkan next ke head lama dan prev head lama ke node baru.
 - Hasil: node baru ditambahkan di awal list.
- c. Fungsi insertEnd_1020
- Membuat node baru dengan data.
 - Jika list kosong, node baru menjadi head.
 - Jika tidak kosong, traversal dilakukan sampai node terakhir, lalu node baru dihubungkan di akhir.
 - Hasil: node baru ditambahkan di akhir list.
- d. Fungsi insertAtPosition_1020
- Membuat node baru dengan data.
 - Jika posisi = 1, node baru langsung menjadi head.
 - Jika posisi > 1, traversal dilakukan sampai posisi yang ditentukan.
 - Node baru dihubungkan di posisi tersebut dengan mengatur prev dan next.
 - Jika posisi tidak valid, ditampilkan pesan “Posisi tidak ada”.
 - Hasil: node baru ditambahkan di posisi tertentu.
- e. Fungsi printList_1020
- Melakukan traversal dari head ke akhir list.
 - Mencetak data setiap node dengan format data <->.
 - Hasil: menampilkan isi DLL secara berurutan.
- f. Fungsi main
- Membuat DLL awal: 2 <-> 3 <-> 5.
 - Mencetak DLL awal.
 - Menambahkan node 1 di awal.
 - Menambahkan node 6 di akhir.
 - Menambahkan node 4 di posisi ke-4.

- Setiap operasi dicetak hasilnya sehingga terlihat perubahan list.

5. Output

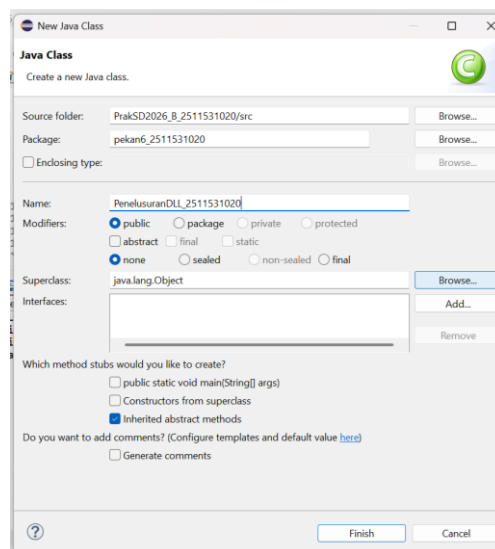


```
<terminated> InsertDLL_2511531020 [Java Application] C:\Users\USER\p2\pool\plugins\org.
DLL Awal: 2 <-> 3 <-> 5 <->
Simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
Simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
Tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->
```

Gambar 2.8 Output

2.2.5 Class 3 “PenelusuranDLL_2511531020”

1. Membuat package pekan6_2511531020 dan class PenelusuranDLL_2511532015



Gambar 2.9 Membuat Nama package dan class 3
PenelusuranDLL_2511532015

2. Klik Finish
3. Kode Program
4. `package pekan6_2511531020;`
- 5.
6. `public class PenelusuranDLL_2511531020 {`
7. `// fungsi penelusuran maju`
8. `static void forwardTraversal_1020(NodeDLL_2511531020`
 `head_1020) {`

```

9.         NodeDLL_2511531020 curr_1020 = head_1020;
10.        while (curr_1020 != null) {
11.            System.out.print(curr_1020.data_1020 + " <-> ");
12.            curr_1020 = curr_1020.next_1020;
13.        }
14.        System.out.println();
15.    }
16.
17.    // fungsi penelusuran mundur
18.    static void backwardTraversal_1020(NodeDLL_2511531020
tail_1020) {
19.        NodeDLL_2511531020 curr_1020 = tail_1020;
20.        while (curr_1020 != null) {
21.            System.out.print(curr_1020.data_1020 + " <-> ");
22.            curr_1020 = curr_1020.prev_1020;
23.        }
24.        System.out.println();
25.    }
26.
27.    public static void main(String[] args) {
28.        // membuat dll 1 <-> 2 <-> 3
29.        NodeDLL_2511531020 head_1020 = new
NodeDLL_2511531020(1);
30.        NodeDLL_2511531020 second_1020 = new
NodeDLL_2511531020(2);
31.        NodeDLL_2511531020 third_1020 = new
NodeDLL_2511531020(3);
32.
33.        head_1020.next_1020 = second_1020;
34.        second_1020.prev_1020 = head_1020;
35.        second_1020.next_1020 = third_1020;
36.        third_1020.prev_1020 = second_1020;
37.
38.        System.out.println("Penelusuran maju: ");
39.        forwardTraversal_1020(head_1020);
40.
41.        System.out.println("Penelusuran mundur: ");
42.        backwardTraversal_1020(third_1020);
43.    }
44. }

```

Gambar 2.10 Class 3 “PenelusuranDLL_2511531020”

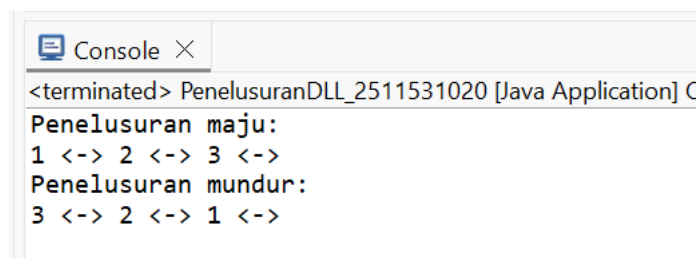
4. Analisis Langkah

a. Deklarasi Package dan Class

- package pekan6_2511531020; → menandakan file berada dalam package sesuai NIM.

- `public class PenelusuranDLL_2511531020` → kelas ini berisi fungsi untuk menelusuri Doubly Linked List (DLL).
- b. Fungsi `forwardTraversal_1020`
- Parameter: `head_1020` (simpul pertama).
 - Proses: mulai dari `head`, cetak `data_1020`, lalu bergerak ke `next_1020` hingga simpul terakhir.
 - Output: menampilkan isi DLL dari depan ke belakang.
- c. Fungsi `backwardTraversal_1020`
- Parameter: `tail_1020` (simpul terakhir).
 - Proses: mulai dari `tail`, cetak `data_1020`, lalu bergerak ke `prev_1020` hingga simpul pertama.
 - Output: menampilkan isi DLL dari belakang ke depan.
- d. Fungsi `main`
- Membuat DLL sederhana dengan node `1 <-> 2 <-> 3`.
 - Menghubungkan pointer `next_1020` dan `prev_1020` agar node saling terhubung dua arah.
 - Memanggil `forwardTraversal_1020` untuk menelusuri maju.
 - Memanggil `backwardTraversal_1020` untuk menelusuri mundur.

4. Output



```

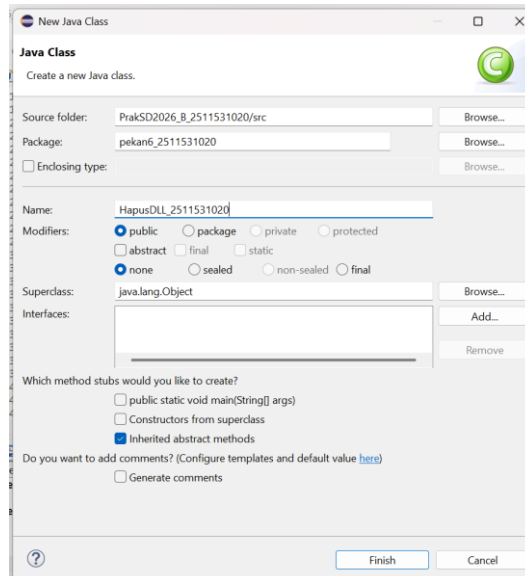
Console X
<terminated> PenelusuranDLL_2511531020 [Java Application] C
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->

```

Gambar 2.11 Membuat Nama package dan class
PenelusuranDLL__2511531020

2.2.6 Class 4 “HapusDLL_2511531020”

1. Membuat package pekan6_2511531020 dan class HapusDLL_2511531020



Gambar 2.12 Membuat Nama package dan class HapusDLL_2511531020

2. Klik Finish
3. Kode Program
4. **package** pekan6_2511531020;
- 5.
6. **public class** HapusDLL_2511531020 {
7. // fungsi menghapus node di awal
8. **public static** NodeDLL_2511531020
9. delHead_1020(NodeDLL_2511531020 head_1020) {
10. **if** (head_1020 == **null**) {
11. **return null**;
12. }
13. NodeDLL_2511531020 temp_1020 = head_1020;
14. head_1020 = head_1020.next_1020;
15. **if** (head_1020 != **null**) {
16. head_1020.prev_1020 = **null**;
17. }
18. **return** head_1020;
19. }
20. // fungsi menghapus node di akhir
21. **public static** NodeDLL_2511531020
22. delLast_1020(NodeDLL_2511531020 head_1020) {
23. **if** (head_1020 == **null**) {
24. **return null**;

```

24.     }
25.     if (head_1020.next_1020 == null) {
26.         return null;
27.     }
28.     NodeDLL_2511531020 curr_1020 = head_1020;
29.     while (curr_1020.next_1020 != null) {
30.         curr_1020 = curr_1020.next_1020;
31.     }
32.     if (curr_1020.prev_1020 != null) {
33.         curr_1020.prev_1020.next_1020 = null;
34.     }
35.     return head_1020;
36. }
37.
38. // fungsi menghapus node di posisi tertentu
39. public static NodeDLL_2511531020
delPos_1020(NodeDLL_2511531020 head_1020, int pos_1020) {
40.     if (head_1020 == null) {
41.         return head_1020;
42.     }
43.     NodeDLL_2511531020 curr_1020 = head_1020;
44.     for (int i_1020 = 1; curr_1020 != null && i_1020 <
pos_1020; ++i_1020) {
45.         curr_1020 = curr_1020.next_1020;
46.     }
47.     if (curr_1020 == null) {
48.         return head_1020;
49.     }
50.     if (curr_1020.prev_1020 != null) {
51.         curr_1020.prev_1020.next_1020 =
curr_1020.next_1020;
52.     }
53.     if (curr_1020.next_1020 != null) {
54.         curr_1020.next_1020.prev_1020 =
curr_1020.prev_1020;
55.     }
56.     if (head_1020 == curr_1020) {
57.         head_1020 = curr_1020.next_1020;
58.     }
59.     return head_1020;
60. }
61.
62. // fungsi mencetak DLL
63. public static void printList_1020(NodeDLL_2511531020
head_1020) {
64.     NodeDLL_2511531020 curr_1020 = head_1020;
65.     while (curr_1020 != null) {
66.         System.out.print(curr_1020.data_1020 + " <-> ");
67.         curr_1020 = curr_1020.next_1020;
68.     }
69.     System.out.println();
70. }
71.
72. public static void main(String[] args) {
73.     NodeDLL_2511531020 head_1020 = new
NodeDLL_2511531020(1);
74.     head_1020.next_1020 = new NodeDLL_2511531020(2);

```

```

75.         head_1020.next_1020.prev_1020 = head_1020;
76.         head_1020.next_1020.next_1020 = new
NodeDLL_2511531020(3);
77.         head_1020.next_1020.next_1020.prev_1020 =
head_1020.next_1020;
78.         head_1020.next_1020.next_1020.next_1020 = new
NodeDLL_2511531020(4);
79.         head_1020.next_1020.next_1020.next_1020.prev_1020 =
head_1020.next_1020.next_1020;
80.         head_1020.next_1020.next_1020.next_1020.next_1020 =
new NodeDLL_2511531020(5);
81.         head_1020.next_1020.next_1020.next_1020.next_1020.prev_1020 =
head_1020.next_1020.next_1020.next_1020;
82.
83.         System.out.print("DLL Awal: ");
84.         printList_1020(head_1020);
85.
86.         System.out.print("Setelah head dihapus: ");
87.         head_1020 = delHead_1020(head_1020);
88.         printList_1020(head_1020);
89.
90.         System.out.print("Setelah node terakhir dihapus: ");
91.         head_1020 = delLast_1020(head_1020);
92.         printList_1020(head_1020);
93.
94.         System.out.print("Menghapus node ke-2: ");
95.         head_1020 = delPos_1020(head_1020, 2);
96.         printList_1020(head_1020);
97.     }
98. }

```

Gambar 2.13 Class 4 “HapusDLL_2511531020”

4. Analisis Langkah

a. Deklarasi Package dan Class

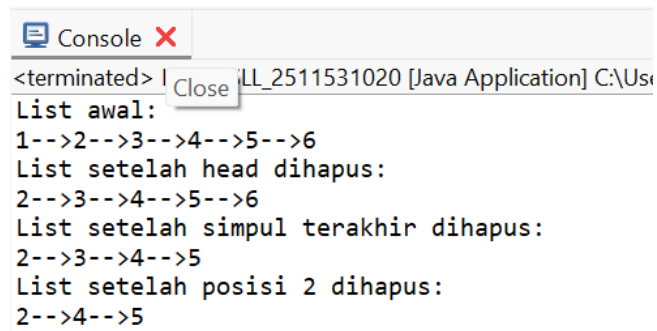
- package pekan6_2511531020; → menandakan file berada dalam package sesuai NIM.
- public class HapusDLL_2511531020 → kelas ini berisi fungsi untuk menghapus node pada Doubly Linked List (DLL).

b. Fungsi delHead_1020

- Mengecek apakah list kosong.
- Jika tidak kosong, head bergeser ke node berikutnya (head_1020.next_1020).

- Pointer prev_1020 dari head baru diatur ke null.
 - Hasil: simpul pertama (head) dihapus.
- c. Fungsi delLast_1020
- Mengecek apakah list kosong atau hanya berisi satu node.
 - Traversal dilakukan sampai node terakhir.
 - Pointer next_1020 dari node sebelumnya diatur ke null.
 - Hasil: simpul terakhir dihapus.
- d. Fungsi delPos_1020
- Mengecek apakah list kosong.
 - Traversal dilakukan sampai posisi yang diminta.
 - Jika node ditemukan, pointer next_1020 dari node sebelumnya dan pointer prev_1020 dari node sesudahnya dihubungkan ulang.
 - Jika node yang dihapus adalah head, maka head bergeser ke node berikutnya.
 - Hasil: simpul pada posisi tertentu dihapus.
- e. Fungsi printList_1020
- Traversal dari head ke tail.
 - Mencetak data setiap node dengan format rapi: "data <->" hanya di antara node, tidak di ujung.
 - Hasil: tampilan list lebih bersih.
- f. Fungsi main
- Membuat DLL awal: 1 <-> 2 <-> 3 <-> 4 <-> 5.
 - Mencetak DLL awal.
 - Menghapus head → hasil: 2 <-> 3 <-> 4 <-> 5.
 - Menghapus node terakhir → hasil: 2 <-> 3 <-> 4.
 - Menghapus node ke-2 → hasil: 2 <-> 4.

5. Output



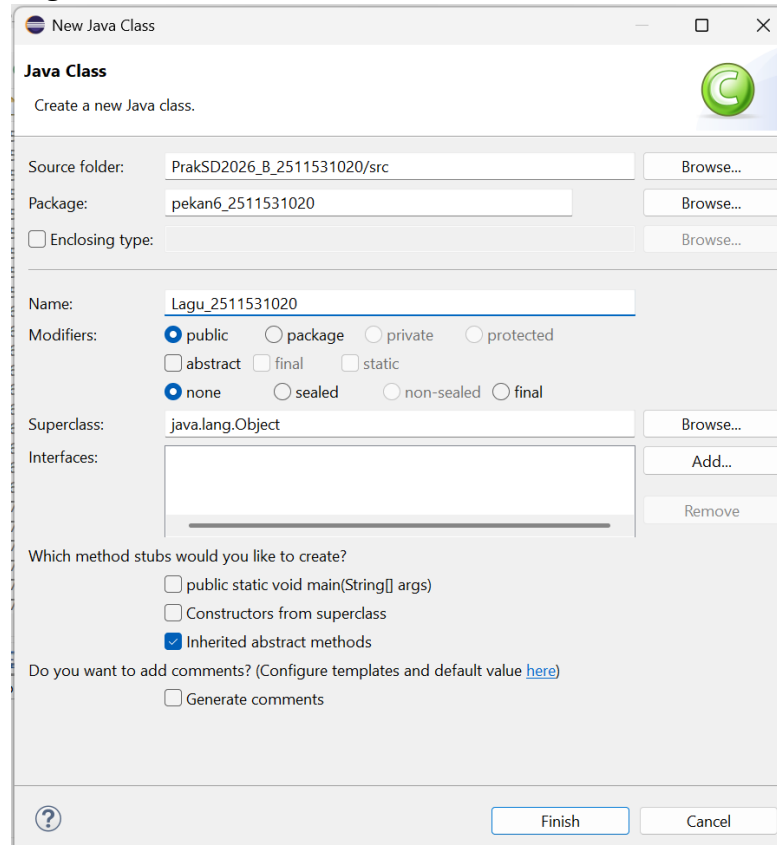
```
<terminated> | LL_2511531020 [Java Application] C:\Usr
List awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir dihapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5
```

Gambar 2.14 Output

2.3 Tugas Praktikum

2.3.1 Class 1 “Lagu_2511531020”

1. Membuat Nama package pekan6_2511531020 dan class Lagu_2511531020



Gambar 2.15 package pekan6_2511531020 dan class Lagu_2511531020

2. Klik Finish

3. Kode Program

```
2. package pekan6_2511531020;
3.
4. public class Lagu_2511531020 {
5.     private String judul_1020;
6.     private String penyanyi_1020;
7.     public Lagu_2511531020 next_1020;
8.     public Lagu_2511531020 prev_1020;
9.
10.    // Constructor
11.    public Lagu_2511531020(String judul_1020, String
    penyanyi_1020) {
12.        this.judul_1020 = judul_1020;
13.        this.penyanyi_1020 = penyanyi_1020;
14.        this.next_1020 = null;
15.        this.prev_1020 = null;
16.    }
17.
18.    // Getter
19.    public String getJudul_1020() {
20.        return judul_1020;
21.    }
22.
23.    public String getPenyanyi_1020() {
24.        return penyanyi_1020;
25.    }
26.
27.    // Setter
28.    public void setJudul_1020(String judul_1020) {
29.        this.judul_1020 = judul_1020;
30.    }
31.
32.    public void setPenyanyi_1020(String
    penyanyi_1020) {
33.        this.penyanyi_1020 = penyanyi_1020;
34.    }
35. }
```

Gambar 2.16 Class “Lagu_2511531020”

4. Analisis Langkah

a. Deklarasi Atribut

- head_1020 → menunjuk ke lagu pertama dalam playlist.
- tail_1020 → menunjuk ke lagu terakhir dalam playlist.

Dua pointer ini penting untuk mengelola playlist berbasis Double Linked List (DLL).

b. Tambah Lagu (Insert at Tail)

- Membuat node lagu baru dengan judul dan penyanyi.
- Jika list kosong → lagu baru langsung jadi head_1020 dan tail_1020.
- Jika tidak kosong → hubungkan tail_1020.next_1020 ke lagu baru, lalu set laguBaru.prev_1020 = tail_1020.
- Update tail_1020 ke node baru.

c. Hapus Lagu Pertama (Delete Head)

- Mengecek apakah playlist kosong.
- Jika tidak kosong → tampilkan judul lagu pertama, lalu geser head_1020 ke node berikutnya.
- Jika head_1020 tidak null → set head_1020.prev_1020 = null.
- Jika playlist kosong setelah penghapusan → tail_1020 juga di-set null.

d. Tampil Maju (Display Forward)

- Traversal dari head_1020 hingga null.
- Cetak semua data lagu (judul dan penyanyi).

e. Tampil Mundur (Display Backward)

- Traversal dari tail_1020 hingga null.
- Cetak semua data lagu dari belakang ke depan.
- Inilah fitur khas Double Linked List yang tidak bisa dilakukan di Single Linked List.

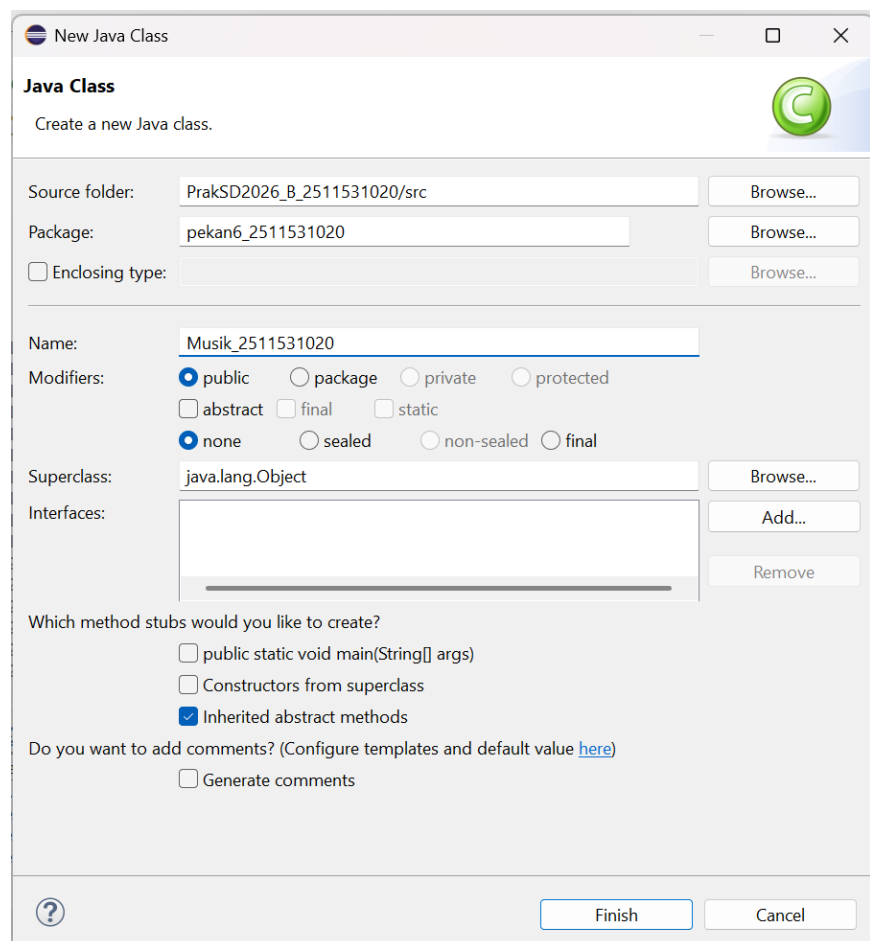
f. Cari Lagu (Search)

- Traversal dari head_1020.
- Bandingkan judul lagu dengan input (case-insensitive).

- Jika ketemu → tampilkan data lagu.
 - Jika tidak → tampilkan pesan “Lagu tidak ditemukan”.
- g. Main Method (Menu Interaktif)
- Menyediakan menu: tambah lagu, hapus lagu pertama, tampil maju, tampil mundur, cari lagu, keluar.
 - Menggunakan Scanner untuk input dari user.
 - Switch-case dipakai untuk memilih operasi sesuai input.

2.3.2 Class 2 “Musik_2511531020”

1. Membuat Nama package pekan6_2511531020 dan class Musik_2511531020



Gambar 2.17 Membuat Nama package dan class Musik_2511531020

2. Klik Finish
3. Kode Program

```

2. package pekan6_2511531020;
3.
4. import java.util.Scanner;
5.
6. public class Musik_2511531020 {
7.     private Lagu_2511531020 head_1020;
8.     private Lagu_2511531020 tail_1020;
9.
10.    // 1. Tambah Lagu
11.    public void tambahLagu_1020(String judul_1020, String
    penyanyi_1020) {
12.        Lagu_2511531020 laguBaru_1020 = new
    Lagu_2511531020(judul_1020, penyanyi_1020);
13.        if (head_1020 == null) {
14.            head_1020 = tail_1020 = laguBaru_1020;
15.        } else {
16.            tail_1020.next_1020 = laguBaru_1020;
17.            laguBaru_1020.prev_1020 = tail_1020;
18.            tail_1020 = laguBaru_1020;
19.        }
20.        System.out.println("Lagu berhasil ditambahkan!");
21.    }
22.
23.    // 2. Hapus Lagu Pertama
24.    public void hapusLaguAwal_1020() {
25.        if (head_1020 == null) {
26.            System.out.println("Playlist kosong!");
27.            return;
28.        }
29.        System.out.println("Menghapus: " +
    head_1020.getJudul_1020());
30.        head_1020 = head_1020.next_1020;
31.        if (head_1020 != null) {
32.            head_1020.prev_1020 = null;
33.        } else {
34.            tail_1020 = null;
35.        }
36.    }
37.
38.    // 3. Tampil Maju
39.    public void tampilMaju_1020() {
40.        if (head_1020 == null) {
41.            System.out.println("Playlist kosong!");
42.            return;
43.        }
44.        Lagu_2511531020 temp_1020 = head_1020;
45.        while (temp_1020 != null) {
46.            System.out.println(temp_1020.getJudul_1020()
    + " - " + temp_1020.getPenyanyi_1020());
47.            temp_1020 = temp_1020.next_1020;
48.        }
49.    }
50.

```

```

51. // 4. Tampil Mundur
52. public void tampilMundur_1020() {
53.     if (tail_1020 == null) {
54.         System.out.println("Playlist kosong!");
55.         return;
56.     }
57.     Lagu_2511531020 temp_1020 = tail_1020;
58.     while (temp_1020 != null) {
59.         System.out.println(temp_1020.getJudul_1020()
+ " - " + temp_1020.getPenyanyi_1020());
60.         temp_1020 = temp_1020.prev_1020;
61.     }
62. }
63.
64. // 5. Cari Lagu
65. public void cariLagu_1020(String judul_1020) {
66.     if (head_1020 == null) {
67.         System.out.println("Playlist kosong!");
68.         return;
69.     }
70.     Lagu_2511531020 temp_1020 = head_1020;
71.     boolean ditemukan_1020 = false;
72.     while (temp_1020 != null) {
73.         if
(temp_1020.getJudul_1020().equalsIgnoreCase(judul_1020))
{
74.             System.out.println("Lagu ditemukan: " +
temp_1020.getJudul_1020() + " - " +
temp_1020.getPenyanyi_1020());
75.             ditemukan_1020 = true;
76.             break;
77.         }
78.         temp_1020 = temp_1020.next_1020;
79.     }
80.     if (!ditemukan_1020) {
81.         System.out.println("Lagu tidak ditemukan!");
82.     }
83. }
84.
85. // Main dengan menu interaktif
86. public static void main(String[] args) {
87.     Musik_2511531020 playlist_1020 = new
Musik_2511531020();
88.     Scanner sc_1020 = new Scanner(System.in);
89.     int pilihan_1020;
90.
91.     do {
92.         System.out.println("\n== Playlist Musik NIM:
2511531020 ==");
93.         System.out.println("1. Tambah Lagu");
94.         System.out.println("2. Hapus Lagu Pertama");
95.         System.out.println("3. Lihat Playlist
(Maju)");
96.         System.out.println("4. Lihat Playlist
(Mundur)");
97.         System.out.println("5. Cari Lagu");
98.         System.out.println("6. Keluar");

```

```

99.         System.out.print("Pilihan: ");
100.         pilihan_1020 = sc_1020.nextInt();
101.         sc_1020.nextLine();
102.
103.         switch (pilihan_1020) {
104.             case 1:
105.                 System.out.print("Judul: ");
106.                 String judul_1020 =
                    sc_1020.nextLine();
107.                 System.out.print("Penyanyi: ");
108.                 String penyanyi_1020 =
                    sc_1020.nextLine();
109.                 playlist_1020.tambahLagu_1020(judul_1020, penyanyi_1020);
110.                 break;
111.             case 2:
112.                 playlist_1020.hapusLaguAwal_1020();
113.                 break;
114.             case 3:
115.                 playlist_1020.tampilMaju_1020();
116.                 break;
117.             case 4:
118.                 playlist_1020.tampilMundur_1020();
119.                 break;
120.             case 5:
121.                 System.out.print("Masukkan judul
                    lagu: ");
122.                 String cari_1020 =
                    sc_1020.nextLine();
123.                 playlist_1020.cariLagu_1020(cari_1020);
124.                 break;
125.             case 6:
126.                 System.out.println("Keluar dari
                    program...");
127.                 break;
128.             default:
129.                 System.out.println("Pilihan
                    tidak valid!");
130.         }
131.     } while (pilihan_1020 != 6);
132.
133.     sc_1020.close();
134. }
135. }

```

Gambar 2.18 Class “Musik_2511531020”

4. Analisis Langkah

a. Deklarasi Atribut (Class Musik_2511531020)

- head_1020 → menunjuk ke lagu pertama dalam playlist.

- `tail_1020` → menunjuk ke lagu terakhir dalam playlist. Dua pointer ini menjaga struktur playlist agar bisa ditelusuri maju dan mundur.
- b. Tambah Lagu (Insert at Tail)
- Membuat node lagu baru (`Lagu_2511531020`) dengan judul dan penyanyi.
 - Jika playlist kosong → node baru jadi `head_1020` sekaligus `tail_1020`.
 - Jika tidak kosong → hubungkan `tail_1020.next_1020` ke node baru, lalu set `laguBaru.prev_1020 = tail_1020`.
 - Update `tail_1020` ke node baru. Lagu baru selalu ditambahkan di akhir playlist.
- c. Hapus Lagu Pertama (Delete Head)
- Mengecek apakah playlist kosong.
 - Jika tidak kosong → tampilkan judul lagu pertama, lalu geser `head_1020` ke node berikutnya.
 - Jika `head_1020` tidak null → set `head_1020.prev_1020 = null`.
 - Jika playlist kosong setelah penghapusan → `tail_1020` juga di-set null. Operasi ini memastikan node pertama dihapus dengan benar.
- d. Tampil Maju (Display Forward)
- Traversal dari `head_1020` hingga null.
 - Cetak semua data lagu (judul dan penyanyi). Menampilkan playlist dari awal ke akhir.
- e. Tampil Mundur (Display Backward)
- Traversal dari `tail_1020` hingga null.
 - Cetak semua data lagu dari belakang ke depan. Fitur khas Double Linked List, karena bisa menelusuri playlist mundur.
- f. Cari Lagu (Search)
- Traversal dari `head_1020`.
 - Bandingkan judul lagu dengan input (case-insensitive).
 - Jika ketemu → tampilkan data lagu.

- Jika tidak → tampilkan pesan “Lagu tidak ditemukan”.
Memudahkan pengguna mencari lagu tertentu dalam playlist.

g. Main Method (Menu Interaktif)

- Menyediakan menu: tambah lagu, hapus lagu pertama, tampil maju, tampil mundur, cari lagu, keluar.
- Menggunakan Scanner untuk input dari user.
- Switch-case dipakai untuk memilih operasi sesuai input. Program bisa dijalankan interaktif sesuai contoh di file tugas.

5. Output

```

Console X
<terminated> Musik_2511531020 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj.open...

=== Playlist Musik NIM: 2511531020 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: Cincin
Penyanyi: Hindia
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511531020 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
Cincin - Hindia

=== Playlist Musik NIM: 2511531020 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4
Cincin - Hindia

```

```
=== Playlist Musik NIM: 2511531020 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu: cincin
Lagu ditemukan: Cincin - Hindia

=== Playlist Musik NIM: 2511531020 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Menghapus: Cincin

=== Playlist Musik NIM: 2511531020 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
Playlist kosong!

=== Playlist Musik NIM: 2511531020 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 6
Keluar dari program...
```

Gambar 2.19 Output

BAB III

KESIMPULAN

Praktikum mengenai Doubly Linked List (DLL) memberikan pemahaman mendalam tentang bagaimana struktur data dinamis bekerja dengan pointer dua arah (next dan prev). Melalui implementasi operasi dasar seperti penambahan node (insert), penghapusan node (delete), serta penelusuran maju dan mundur (traversal), mahasiswa dapat melihat secara langsung fleksibilitas DLL dibandingkan dengan Single Linked List.

DLL memungkinkan manipulasi data lebih efisien karena simpul dapat diakses dari kedua arah, sehingga proses penghapusan maupun penambahan node di berbagai posisi dapat dilakukan tanpa harus membangun ulang seluruh list. Praktikum ini juga menegaskan pentingnya pemahaman pointer dalam pemrograman, karena kesalahan kecil pada penghubungan next dan prev dapat menyebabkan list terputus atau output tidak sesuai.

Praktikum ini melatih keterampilan analisis, logika pemrograman, serta memberikan dasar yang kuat untuk mempelajari struktur data yang lebih kompleks seperti circular linked list, stack, queue, maupun tree.

DAFTAR PUSTAKA

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.
- [2] E. Horowitz, S. Sahni, and D. Mehta, *Fundamentals of Data Structures in C*, 2nd ed. New Delhi: Universities Press, 2008.
- [3] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Boston, MA: Addison-Wesley, 2011.
- [4] Oracle, “The Java™ Tutorials: Collections and Data Structures,” Oracle, 2024. [Online]. Available: <https://docs.oracle.com/javase/tutorial/>
- [5] W. Stallings, *Data and Computer Communications*, 10th ed. Upper Saddle River, NJ: Pearson, 2013.